

UNIVERSITÀ DEGLI STUDI DI BRESCIA

Corso di Programmazione Full Stack

Sistema di pianificazione degli appelli

Documentazione tecnica del progetto

Monorepo full-stack — NestJS, React, PostgreSQL, TypeORM

Autore: Gabriele Fiorucci

25 giugno 2026

Indice

1	Introduzione e dominio applicativo	2
1.1	Scopo del documento	2
1.2	Il dominio: la pianificazione degli appelli	2
1.3	Gli attori e i loro permessi	3
1.4	Regole di dominio principali	3
1.5	Struttura del documento	4
2	Architettura del sistema e stack tecnologico	5
2.1	Visione d'insieme	5
2.2	Perché un monorepo Nx	5
2.3	Applicazioni e librerie	6
2.4	Lo stack tecnologico	6
2.5	La metodologia WebML	6
2.6	Project references e <code>customConditions</code>	6
3	Progettazione secondo la metodologia WebML	8
3.1	Specifica dei requisiti: gruppi utente e casi d'uso	8
3.2	Dalla matrice CRUD alle rotte e alle «guardie»	8
3.3	Progettazione dei dati	9
3.4	Progettazione dell'ipertesto	10
4	Backend: avvio, configurazione e modello dei dati	13
4.1	Il bootstrap dell'applicazione (<code>main.ts</code>)	13
4.2	Composizione del modulo radice (<code>AppModule</code>)	14
4.3	La configurazione del database (<code>DatabaseModule</code>)	14
4.4	Il modello dei dati	15
4.4.1	Convenzioni comuni	15
4.4.2	User, Teacher, Secretariat	16
4.4.3	DegreeCourse e Subject	16
4.4.4	ExamSession ed Exam	17
4.5	Lo script di <i>seed</i>	17
5	Sicurezza e autenticazione	18
5.1	La libreria <code>@server/security</code>	18
5.2	La libreria <code>@server/auth</code>	19
5.3	La libreria <code>@server/users</code>	20
5.4	Il flusso di autenticazione e la policy di onboarding	20

6	Il pattern a livelli e la logica di dominio	21
6.1	Il pattern a quattro livelli	21
6.2	Il barrel <code>index.ts</code> e la condivisione col frontend	22
6.3	I DTO: validazione e trasformazione dell'input	22
6.4	La gestione centralizzata degli errori del database	23
6.5	Le rotte REST del dominio	24
6.6	Logica per entità	24
6.6.1	Corsi di laurea	24
6.6.2	Insegnamenti	24
6.6.3	Sessioni d'esame	25
6.6.4	Appelli (la logica più ricca)	25
6.6.5	Docenti e segretari (creazione atomica e <i>self-only</i>)	25
7	Il frontend React	27
7.1	Organizzazione del codice	27
7.2	Avvio e routing	27
7.3	L'infrastruttura HTTP condivisa	28
7.4	L'autenticazione lato client	28
7.5	Il layout e la navbar	29
7.6	Il pattern api/page in dettaglio	29
7.7	I form di creazione e modifica	30
7.8	La dashboard e il calendario	31
7.9	Note sullo stato e sulla cache	32
8	Test e qualità del codice	33
8.1	Unit test dei service	33
8.2	Smoke test dei controller	33
8.3	Test d'integrazione ed end-to-end	34
8.4	Esecuzione	34
9	Interazione end-to-end, WebML e procedure	35
9.1	Una richiesta dal clic alla risposta	35
9.2	La mappatura WebML → React	36
9.3	Comandi per build, avvio e test	36
9.4	Conclusioni	37
A	Considerazioni di produzione	38
A.1	Schema del database: da <code>synchronize</code> alle migrazioni	38
A.2	Gestione dei segreti	38
A.3	Autenticazione: scadenza e conservazione del token	38
A.4	Trasporto, intestazioni e CORS	38
A.5	Esposizione della documentazione API	39
A.6	Build, esercizio e osservabilità	39
A.7	Riepilogo	39

Capitolo 1

Introduzione e dominio applicativo

1.1 Scopo del documento

Questo documento descrive in modo accurato e completo l'intero progetto: la sua **struttura**, l'**architettura**, le **scelte di progettazione** e il **codice** che lo compone. L'obiettivo è offrire un *walkthrough* che permetta di capire come è costruito il sistema, quali tecniche e pattern sono stati adottati, come sono organizzate le rotte HTTP e come interagiscono fra loro le componenti del backend, quelle del frontend e i due lati dell'applicazione.

L'applicazione è stata costruita seguendo le regole e le metodologie viste nel corso: la progettazione dei dati e dell'ipertesto con il metodo **WebML**, i pattern di **NestJS** per il backend, **TypeORM** per la persistenza, l'autenticazione con **JWT + Passport** e **React** per il frontend. Dove utile, il testo richiama esplicitamente la regola del corso che ha guidato una scelta.

1.2 Il dominio: la pianificazione degli appelli

Il sistema modella la *pianificazione degli appelli d'esame* di un ateneo. Gli oggetti del dominio sono:

- **Corso di laurea** (*degree course*): ha un nome, una durata in anni e un dipartimento. Raggruppa gli insegnamenti.
- **Insegnamento** / materia (*subject*): appartiene a un corso di laurea, è collocato in un anno di corso, vale un certo numero di **CFU** ed è tenuto da un docente.
- **Sessione d'esame** (*exam session*): un periodo dell'anno in cui si svolgono gli appelli (es. "Sessione Estiva 2026"). Definisce due intervalli temporali distinti: il periodo in cui gli appelli possono *tenersi* e la *finestra di pianificazione* entro cui i docenti possono inserirli.
- **Appello** / esame (*exam*): la singola prova d'esame, con data, fascia oraria, tipo (orale/scritto) e tipo d'aula richiesto. Collega una materia, una sessione e il docente che lo pianifica.
- **Docente** (*teacher*) e **Segretario** (*secretariat*): i due profili utente del sistema, ciascuno collegato a un account (*user*) per l'autenticazione.

Le entità e i campi del dominio usano nomi italiani dove riflettono il linguaggio del dominio (ad es. `yearsDuration`, `cfu`); i messaggi d'errore rivolti all'utente e gli esempi della documentazione API sono in italiano.

1.3 Gli attori e i loro permessi

Il sistema riconosce due ruoli, modellati dall'enumerazione `UserRole`:

Segreteria

Agisce da *amministratore*. Gestisce (crea, modifica, elimina) corsi di laurea, insegnamenti, sessioni d'esame, docenti e segretari; vede tutti gli appelli e può modificarli o eliminarli. Registra i nuovi account del sistema.

Docente

Pianifica i propri appelli: può creare un esame solo per le proprie materie e solo quando la finestra di pianificazione della sessione è aperta; può modificare ed eliminare solo i propri appelli, e solo se non sono ancora passati. In sola lettura vede i corsi e gli insegnamenti che lo riguardano.

La tabella seguente sintetizza “chi può fare cosa” sulle risorse principali; i dettagli implementativi (guard, controlli nei service) sono approfonditi nei capitoli 5 e 6.

Risorsa	Segreteria	Docente
Corsi di laurea	CRUD completo	sola lettura (i propri)
Insegnamenti	CRUD completo	sola lettura (i propri)
Sessioni d'esame	CRUD completo	sola lettura
Appelli (esami)	legge tutti, modifica/elimina	crea/modifica/elimina solo i propri
Docenti	CRUD completo	solo il proprio profilo
Segretari	crea/legge	solo il proprio profilo (self-only)

Tabella 1.1: Matrice sintetica dei permessi per ruolo.

1.4 Regole di dominio principali

Oltre ai permessi, il sistema impone una serie di **vincoli di dominio** che il backend fa rispettare in modo autoritativo (il frontend li rispecchia ma non li sostituisce):

- un appello può essere pianificato da un docente solo *dentro* la finestra di pianificazione della sessione;
- la data dell'appello deve cadere nel periodo della sessione e *non* può essere un sabato o una domenica;
- non possono esistere due appelli nello stesso giorno per lo stesso *anno* dello stesso *corso di laurea* (regola di non sovrapposizione “corso+anno+giorno”);
- una stessa materia non può avere più di un appello nella stessa sessione;
- l'anno di un insegnamento deve essere compreso tra 1 e la durata del corso di laurea;
- i periodi di due sessioni d'esame non possono sovrapporsi;
- le entità “padre” (corso, insegnamento, sessione, docente) non sono eliminabili finché esistono entità “figlie” collegate.

1.5 Struttura del documento

Il resto della documentazione è organizzato così: il capitolo 2 presenta l'architettura del monorepo e lo stack tecnologico; il capitolo 3 ripercorre la progettazione secondo la metodologia WebML del corso; il capitolo 4 descrive l'avvio del backend, la configurazione del database e il modello dei dati; il capitolo 5 tratta sicurezza e autenticazione; il capitolo 6 approfondisce il pattern a livelli e la logica di dominio della pianificazione; il capitolo 7 descrive il frontend React; il capitolo 8 tratta i test e la qualità del codice; il capitolo 9 ricostruisce l'interazione end-to-end fra frontend e backend, la mappatura WebML→React e le procedure di build/test.

Capitolo 2

Architettura del sistema e stack tecnologico

2.1 Visione d'insieme

Il progetto è un'applicazione **full-stack** organizzata come **monorepo** gestito con **Nx**. Un unico repository contiene sia il backend sia il frontend, più alcune librerie condivise. I due lati comunicano esclusivamente via **HTTP/JSON**: il frontend è una *Single Page Application* (SPA) che consuma le API REST esposte dal backend.

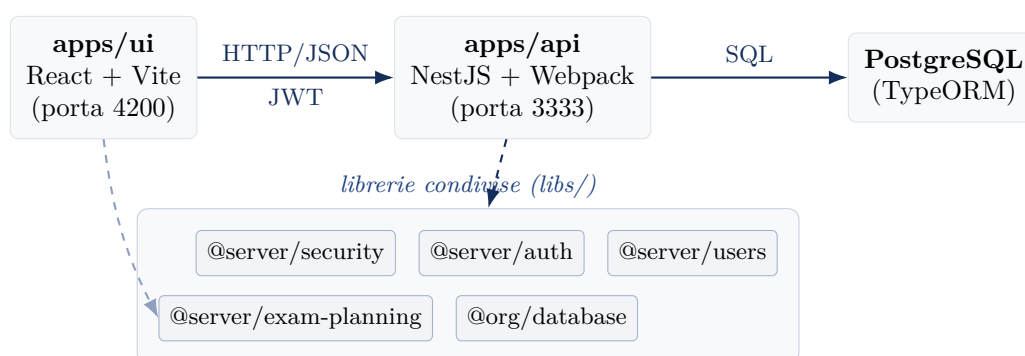


Figura 2.1: Architettura a blocchi: SPA React, API NestJS, database PostgreSQL e librerie condivise del monorepo. Il frontend riusa i *tipi* (`import type`) della libreria di dominio.

2.2 Perché un monorepo Nx

Nx organizza il codice in **progetti** (*apps* e *libs*) all'interno di un solo repository, con un *task runner* che sa costruire, testare e “lintare” ogni progetto e che calcola le dipendenze tra di essi. I vantaggi sfruttati dal progetto sono:

- **condivisione di codice e tipi** tra backend e frontend senza pubblicare pacchetti: la libreria di dominio espone entità, DTO e interfacce che il frontend importa come *tipi*;
- **confini netti** tra moduli: ogni feature del backend è una libreria con un proprio `index.ts` (`barrel`) che ne definisce la superficie pubblica;
- **comandi uniformi**: `nx build`, `nx test`, `nx lint`, `nx serve` funzionano allo stesso modo su ogni progetto.

2.3 Applicazioni e librerie

La tabella seguente elenca i progetti del workspace e il loro ruolo. Ogni libreria è importabile tramite un *alias* (`@server/...`, `@org/...`) invece che con un percorso relativo.

Progetto	Posizione	Ruolo
<code>@org/api</code>	<code>apps/api</code>	Applicazione NestJS (server REST).
<code>@org/ui</code>	<code>apps/ui</code>	SPA React + React Router (Vite).
<code>@org/database</code>	<code>libs/database</code>	Configurazione TypeORM/PostgreSQL.
<code>@server/security</code>	<code>libs/server/security</code>	Guard, decoratori, enum <code>UserRole</code> .
<code>@server/users</code>	<code>libs/server/users</code>	Entità utente + CRUD.
<code>@server/auth</code>	<code>libs/server/auth</code>	Autenticazione JWT + Passport.
<code>@server/exam-planning</code>	<code>libs/server/exam-planning</code>	Dominio della pianificazione appelli.
<code>@org/api-e2e</code>	<code>apps/api-e2e</code>	Test d'integrazione dell'API (Jest).
<code>@org/ui-e2e</code>	<code>apps/ui-e2e</code>	Test end-to-end della UI (Playwright).

Tabella 2.1: Progetti del monorepo.

Il backend (`apps/api`) è volutamente “sottile”: il suo `AppModule` si limita a comporre i moduli di dominio definiti nelle librerie. La logica vera vive nelle `libs/`, secondo l'idea Nx di tenere le *app* come semplici contenitori e le *lib* come unità riusabili e testabili.

2.4 Lo stack tecnologico

2.5 La metodologia WebML

L'intero progetto è stato costruito seguendo la metodologia **WebML** (*Web Modeling Language*) insegnata nel corso, che *guida la codifica* sia del backend sia del frontend a partire da un modello dei dati e dell'ipertesto. Data la sua importanza, alla progettazione WebML è dedicato per intero il capitolo 3, che ne ripercorre i passi (specifica dei requisiti, progettazione dei dati, progettazione dell'ipertesto) e li mappa puntualmente sul codice del progetto.

2.6 Project references e customConditions

Il workspace usa i **project references** di TypeScript in modalità *package-based*: `tsconfig.base.json` dichiara `customConditions: ["@org/source"]` e *non* usa i classici `paths`. Gli array `references` dei vari `tsconfig` sono gestiti **automaticamente** dal generatore `@nx/js:typescript-sync` a partire dagli *import reali* presenti nel codice: per collegare una libreria a un'app basta scrivere un vero `import` e poi eseguire `nx sync` (qualunque target lo innesca). Modifiche manuali agli array `references` verrebbero sovrascritte al successivo `sync`; se un target fallisce con “The workspace is out of sync” si esegue `npx nx sync`.

Ambito	Tecnologia	Uso nel progetto
Linguaggio	TypeScript 5.9	Tipizzazione statica su tutto lo stack.
Backend	NestJS 11	Framework a moduli/DI per l'API REST.
ORM	TypeORM	Mapping entità↔tabelle, query.
Database	PostgreSQL	Persistenza relazionale.
Auth	Passport + JWT	Strategie <i>local</i> e <i>jwt</i> , token Bearer.
Hashing	bcrypt	Hash delle password (salt 10).
Validazione	class-validator / class-transformer	Validazione e trasformazione dei DTO.
Doc API	Swagger (@nestjs/swagger)	UI interattiva su <code>/api/docs</code> .
Frontend	React 19	Componenti, hook, SPA.
Routing FE	React Router 6	Rotte client, rotte protette.
Build FE	Vite 7	Dev server e bundling del frontend.
Build BE	Webpack	Bundling del server NestJS.
Calendario	react-big-calendar + date-fns	Vista mensile della dashboard.
Stile	Tailwind CSS 3	Stile utility-first del frontend.
Monorepo	Nx 22	Task runner, grafo dei progetti.
Test	Jest / Playwright	Unit/integrazione e end-to-end.

Tabella 2.2: Tecnologie principali e loro impiego.

Capitolo 3

Progettazione secondo la metodologia WebML

Il professore insegna che un'applicazione web *data-intensive* non va improvvisata ma **progettata** con la metodologia **WebML**, che guida la codifica del backend e del frontend. I suoi passi sono: *specificazione dei requisiti* (gruppi utente, casi d'uso, dizionario dei dati), *progettazione dei dati* (schema ER/UML), *progettazione dell'ipertesto* (site view, aree, pagine) e infine *implementazione*. Questo capitolo ripercorre tali passi applicati al progetto e mostra come ciascun concetto si traduca nel codice descritto nei capitoli successivi.

3.1 Specifica dei requisiti: gruppi utente e casi d'uso

WebML parte dall'identificazione dei **gruppi utente** (con un *gruppo target* di riferimento) organizzati in una gerarchia con *ereditarietà*. Nel progetto i gruppi sono **DOCENTE** e **SEGRETERIA**, entrambi specializzazioni di un *Utente* autenticato — una gerarchia che si ritrova tale e quale nello schema dei dati (§3.3). La tabella seguente è la scheda dei due gruppi nello stile del corso.

	Segreteria	Docente
Descrizione	amministra l'intero sistema degli appelli	pianifica i propri appelli
Super-gruppo	Utente	Utente
Oggetti lettura	tutte le entità	corsi/insegnamenti propri, sessioni, propri appelli
Oggetti scrittura	corsi, insegnamenti, sessioni, docenti, segretari	propri appelli, proprio profilo
Casi d'uso	«CRUD corsi/insegnamenti/sessioni», «gestione docenti/segretari»	«pianifica appello», «modifica profilo»

Tabella 3.1: Scheda dei gruppi utente del progetto (stile WebML).

3.2 Dalla matrice CRUD alle rotte e alle «guardie»

Il professore mostra che dall'incrocio tra *gruppi utente* e *oggetti* si costruisce la **matrice CRUD**, e che da essa si derivano le “rotte dell'applicazione”: gli oggetti in lettura danno azioni **READ**, gli oggetti in scrittura più i casi d'uso danno azioni **CREATE/UPDATE/DELETE**; soprattutto, *la mappatura tra gruppi utente e casi d'uso definisce le «guardie» sulle rotte*. È esattamente ciò che nel progetto realizzano il **RolesGuard** e il decoratore **@Roles(...)** (cap. 5).

Oggetto	Segreteria	Docente
Corso di laurea	CRUD	R (i propri)
Insegnamento	CRUD	R (i propri)
Sessione d'esame	CRUD	R
Appello	(CUD)R	(CUD)R (i propri)
Docente	CRUD	(CUD)R (sé stesso)
Segretario	CR	(CUD)R (sé stesso)

Tabella 3.2: Matrice CRUD del progetto. Notazione del corso: R sola lettura, CRUD pieno controllo, (CUD)R = legge tutto ma crea/modifica/elimina solo *con limitazioni* (es. solo i propri).

Le *parentesi* della notazione — “con limitazioni, es. solo quelli inseriti da lui” — corrispondono nel progetto ai controlli di *proprietà* applicati nei service: il Docente modifica/elimina solo i propri appelli e il proprio profilo; un Segretario opera solo sul proprio profilo (*self-only*). La matrice è anche uno strumento di verifica: aiuta a scoprire errori di progetto (oggetti che nessuno legge, o che nessuno crea/aggiorna).

3.3 Progettazione dei dati

La modellazione dei dati WebML usa le primitive del **modello ER** (entità, attributi, relazioni con cardinalità, gerarchia *IS-A*) e produce uno **schema ER/UML**. La figura 3.1 è lo schema del progetto.

WebML suggerisce di classificare gli oggetti in quattro tipi di sottoschema; il progetto vi si mappa così:

Core

gli oggetti informativi principali: CORSO DI LAUREA, INSEGNAMENTO, SESSIONE, APPELLO. Sono le entità attorno a cui ruota lo schema.

Personalizzazione

descrive le proprietà degli utenti: i profili DOCENTE e SEGRETERIA sono *sotto-classi IS-A* di UTENTE, esattamente il “sottoschema di personalizzazione” del corso. La linea guida WebML — *inserire l'identificatore dell'utente nelle rotte che estraggono dati collegati all'utente* — è realizzata dalle rotte /mine e /me, dal metodo `findById` dei repository e dal fatto che, creando un appello, il docente è forzato all'utente autenticato.

Interconnessione

le associazioni semantiche tra entità core (*Insegnamento*→*Corso*, *Appello*→*Sessione*, ecc.). Guidano i “link” di navigazione e, lato dati, la regola di buona progettazione applicata è il vincolo `onDelete: RESTRICT`: non si elimina un'entità finché ha entità collegate.

Accesso

gli oggetti ausiliari di categorizzazione/ricerca. Nel progetto sono minimali (es. i filtri per corso/docente sugli insegnamenti).

Una linea guida del corso particolarmente evidente è quella sulle **entità deboli / componenti part-of**: la loro creazione avviene *contestualmente* all'entità core (“fusione delle rotte di creazione”) e la cancellazione della core comporta una cancellazione a cascata. Nel progetto questo è il pattern di creazione **atomica** di UTENTE + profilo (un solo endpoint POST `/teachers` crea entrambi, con *rollback* in caso di errore) e la relazione `onDelete: CASCADE` tra profilo e utente.

3.4 Progettazione dell'ipertesto

La progettazione dell'ipertesto modella il front-end. Il concetto fondamentale è la **site view**: una “vista sul sito” per ciascun gruppo utente. Nel progetto la site view si concretizza nel **rendering condizionato al ruolo**: navbar filtrata (voci `segreteriaOnly`), dashboard diversa per Docente e Segreteria, scelta tra `/mine` e la lista completa. Le aree si dividono in **pubbliche** (il login) e **private** (tutto il resto, dietro `ProtectedRoute`).

Progetto coarse: aree e primitive. Si individuano le *aree* (insiemi coesi di pagine) e le si descrive con le primitive del corso. Le aree del progetto — Esami, Sessioni, Insegnamenti, Corsi di laurea, Docenti, Segretari, Area personale — usano le primitive `Core(...)` per la pubblicazione dei contenuti e `Create/Modify/Delete` per le operazioni; in particolare la creazione di un appello (che collega materia e sessione) e quella di un docente (`UTENTE` + profilo) sono istanze della primitiva `Create&Connect`.

Progetto dettagliato: pagine, visibilità, unità e link. Le *aree* si suddividono in **pagine**, che diventano i componenti React `*.page.tsx`. La *visibilità* delle pagine del corso si ritrova puntualmente:

- **Home** (*H*): la rotta `/` (dashboard col calendario), “su cui si atterra dopo il login”;
- **Landmark** (*L*): le pagine globalmente raggiungibili da ogni altra — sono le voci della **navbar**. Il professore dice esplicitamente che il raggruppamento di pagine landmark “determina la navbar del sito”;
- **Interne**: le pagine raggiungibili solo via link espliciti, come i form `create/edit`.

Le **unità di contenuto** (`Details`, `SimpleList`, `List`, `Form...`) si mappano sui componenti di visualizzazione: una *List* è la tabella di una pagina-lista, un *Form* è una pagina di creazione/modifica. I **link** si distinguono in *non contestuali* (la navbar, che non trasporta contesto) e *contestuali* (che trasportano un parametro): nel progetto il click su un giorno verde del calendario apre il form già precompilato passando i dati in query-string, e il pulsante “Modifica” porta all’edit con l’`:id`. Le primitive di **login/logout** sono le pagine omonime; i **parametri di contesto** (lo “stato” della pagina) sono gestiti con `useSearchParams` e `useParams`.

La mappatura WebML → React. La tabella riassume la corrispondenza adottata (ripresa anche nel capitolo conclusivo).

In sintesi, la struttura del codice non è arbitraria: deriva dal modello WebML. I capitoli seguenti descrivono l’*implementazione* di questo progetto.

Primitiva WebML	Realizzazione React
Site view (per gruppo utente)	rendering condizionato a <code>user.role</code>
Area / pagina landmark	voce della navbar (<code>AppLayout</code>)
Pagina	componente <code>xxx.page.tsx</code>
Home page (H)	rotta / (dashboard)
Pagina interna	form <code>create/edit</code> (link espliciti)
Unità di vista (List/Details)	tabella/elenco nel componente
Unità Form + Operation (CUD)	pagine-form + funzioni di scrittura in <code>*.api.ts</code>
Create&Connect	creazione con FK (es. <code>UTENTE+profilo</code>)
Personalization	rotte <code>/mine</code> , <code>/me</code> ; filtro per utente
Link non contestuale	navbar
Link contestuale / parametri	<code>navigate</code> con <code>query/:id</code> ; <code>useSearchParams/useParams</code>
Login/logout	pagine <code>login/logout</code>
Area pubblica / privata	fuori / dentro <code>ProtectedRoute</code>

Tabella 3.3: Corrispondenza tra primitive WebML e costrutti React del progetto.

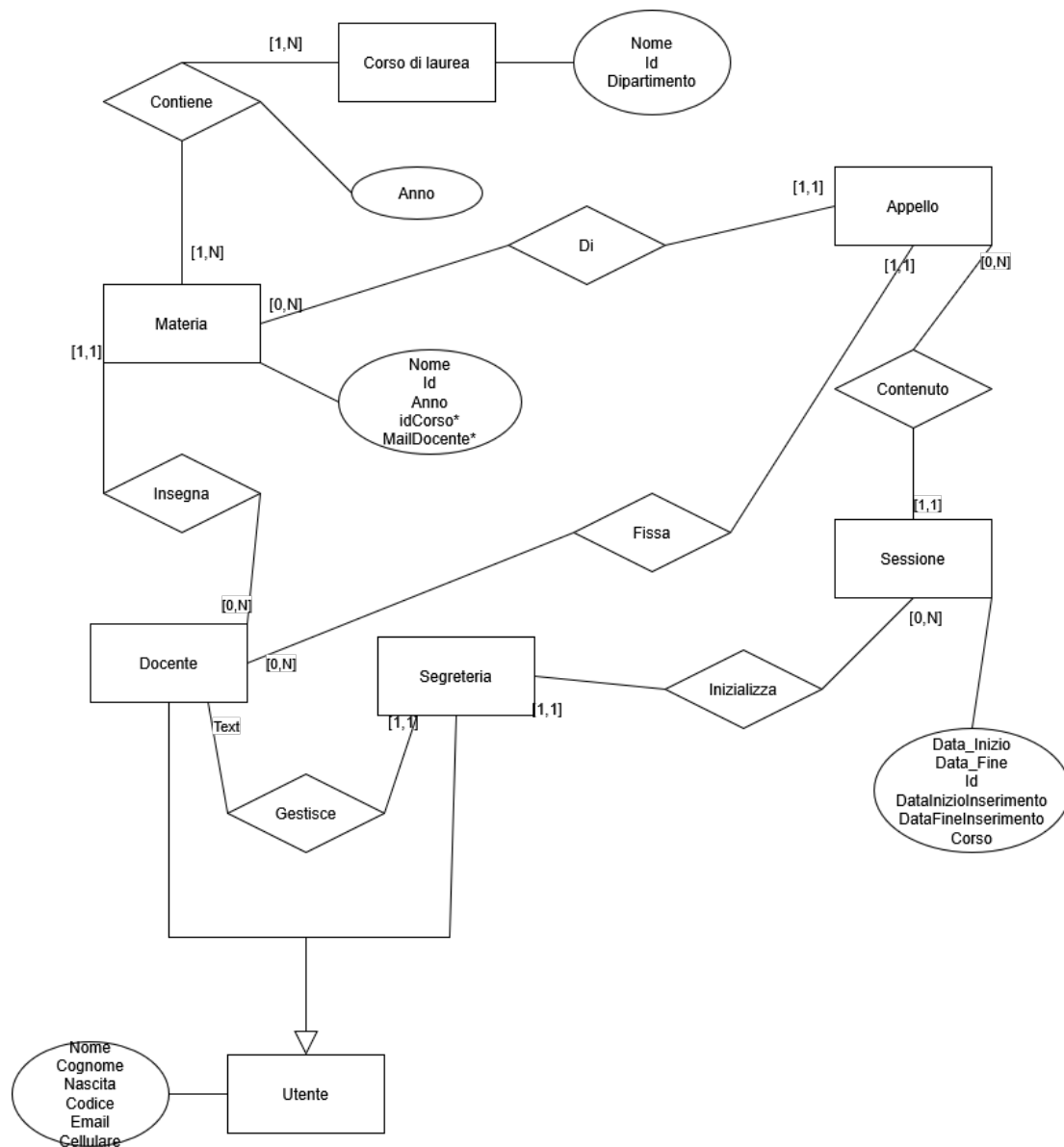


Figura 3.1: Schema dei dati del progetto (ER/UML). Si notino la gerarchia *IS-A* DOCENTE/SEGRETARIA → UTENTE e le relazioni *Contiene*, *Insegna*, *Di*, *Contenuto*, *Fissa*, *Inizializza*, *Gestisce*.

Capitolo 4

Backend: avvio, configurazione e modello dei dati

Questo capitolo descrive come il backend NestJS si avvia, come è configurato il database e come è strutturato il modello dei dati persistente. I capitoli successivi entreranno nella logica applicativa (autenticazione e dominio).

4.1 Il bootstrap dell'applicazione (main.ts)

Il file `apps/api/src/main.ts` è il punto d'ingresso del server: crea l'istanza Nest a partire da `AppModule` e ne configura il comportamento globale prima di mettersi in ascolto.

```
1  async function bootstrap() {
2    const app = await NestFactory.create(AppModule);
3    app.setGlobalPrefix('api'); // tutte le rotte sotto /api
4
5    // Nessuna risposta API deve essere messa in cache dal browser
6    app.use((_req, res, next) => {
7      res.setHeader('Cache-Control', 'no-store');
8      next();
9    });
10
11   app.enableCors({ // il frontend gira su un'altra
12     porta
13     origin: process.env.CORS_ORIGIN ?? 'http://localhost:4200',
14     credentials: true,
15   });
16
17   app.useGlobalPipes(new ValidationPipe({ // validazione+trasformazione DTO
18     whitelist: true, forbidNonWhitelisted: true,
19     transform: true, transformOptions: { enableImplicitConversion: true },
20   }));
21
22   app.useGlobalInterceptors(new
23     ClassSerializerInterceptor(app.get(Reflector)));
24
25   // Swagger su /api/docs
26   const document = SwaggerModule.createDocument(app, config);
27   SwaggerModule.setup('api/docs', app, document);
28
29   await app.listen(process.env.PORT || 3000);
30 }
```

Listing 4.1: Estratto di `apps/api/src/main.ts`.

Le configurazioni globali, con la loro motivazione:

Prefisso globale /api

tutte le rotte sono servite sotto /api (es. /api/auth/login). Tiene separato lo spazio delle API da eventuali risorse statiche.

Cache-Control: no-store

aggiunto a *ogni* risposta. Senza di esso il solo *ETag* di Express potrebbe far riusare al browser una copia vecchia di una lista (ad es. mostrare una sessione già eliminata sul server). È una correzione mirata a un problema osservato di cache.

CORS

abilita le richieste cross-origin dal frontend (porta 4200) verso l'API (porta 3333), poiché sono su origini diverse.

ValidationPipe globale

con `whitelist` (rimuove i campi non dichiarati nei DTO), `forbidNonWhitelisted` (rifiuta con 400 i campi sconosciuti), `transform` e `enableImplicitConversion` (converte automaticamente stringhe in numeri/date secondo il tipo del DTO). È il motore che fa girare i decoratori `class-validator` dei DTO. Le conseguenze: corpi di richiesta “puliti” e tipizzati, e validazione dichiarativa centralizzata.

ClassSerializerInterceptor

applica le regole di `class-transformer` in uscita; in particolare onora il decoratore `@Exclude()` sul campo `passwordHash` dell'utente, così l'hash non finisce mai nelle risposte serializzate.

Swagger

genera la documentazione interattiva delle API, raggiungibile su `http://localhost:<PORT>/api/docs`, con schema *Bearer* per provare le rotte protette.

4.2 Composizione del modulo radice (AppModule)

Coerentemente con la scelta di tenere l'app “sottile”, `AppModule` si limita a importare i moduli di dominio definiti nelle librerie:

```
1 @Module({
2   imports: [ServerUsersModule, DatabaseModule,
3             ServerAuthModule, ServerExamPlanningModule],
4   controllers: [AppController],
5   providers: [AppService],
6 })
7 export class AppModule {}
```

Listing 4.2: `apps/api/src/app/app.module.ts`.

Ogni modulo importato è una libreria autonoma: `DatabaseModule` apre la connessione al database, `ServerUsersModule` e `ServerAuthModule` gestiscono utenti e autenticazione, `ServerExamPlanningModule` raccoglie tutte le entità di dominio. NestJS, tramite la *dependency injection*, risolve le dipendenze tra questi moduli all'avvio.

4.3 La configurazione del database (DatabaseModule)

La libreria `@org/database` configura TypeORM su PostgreSQL leggendo i parametri dal file `.env`. Una piccola funzione `requireEnv` fa fallire l'avvio con un messaggio chiaro se manca una variabile obbligatoria, invece di proseguire con una connessione incompleta.


```

1 function requireEnv(name: string): string {
2   const v = process.env[name];
3   if (!v) throw new Error(`Variabile d'ambiente obbligatoria mancante:
4     ${name}`);
5   return v;
6 }
7
8 @Module({
9   imports: [TypeOrmModule.forRoot({
10     type: 'postgres',
11     host: requireEnv('PG_HOST'),
12     port: Number(requireEnv('PG_PORT')),
13     username: requireEnv('PG_USERNAME'),
14     password: requireEnv('PG_PASSWORD'),
15     database: requireEnv('PG_DATABASE'),
16     autoLoadEntities: true,    // registra le entità dai feature module
17     synchronize: true,        // allinea lo schema al codice (solo sviluppo!)
18   })],
19   exports: [TypeOrmModule],
20 })
21 export class DatabaseModule {}

```

Listing 4.3: `libs/database/src/lib/database.module.ts`.

Due opzioni meritano attenzione:

- `autoLoadEntities: true` — le entità non vanno elencate a mano qui: ogni feature module le registra con `TypeOrmModule.forFeature([...])` e TypeORM le carica automaticamente.
- `synchronize: true` — all'avvio TypeORM allinea lo schema del database alle entità del codice. È comodissimo in sviluppo (niente migrazioni manuali) ma **non va usato in produzione**, perché può alterare le tabelle in modo distruttivo.

Variabile	Significato
PORT	Porta di ascolto dell'API (il <code>.env</code> usa 3333; default 3000).
PG_HOST	Host PostgreSQL.
PG_PORT	Porta PostgreSQL.
PG_USERNAME	Utente del database.
PG_PASSWORD	Password del database.
PG_DATABASE	Nome del database.
SECRET_KEY	Segreto per la firma dei token JWT.

Tabella 4.1: Variabili d'ambiente lette dal `.env`.

4.4 Il modello dei dati

Il dominio è rappresentato da otto entità TypeORM. La figura 4.1 sintetizza le relazioni; le sezioni successive ne descrivono i dettagli.

4.4.1 Convenzioni comuni

Tutte le entità di dominio condividono alcune convenzioni:

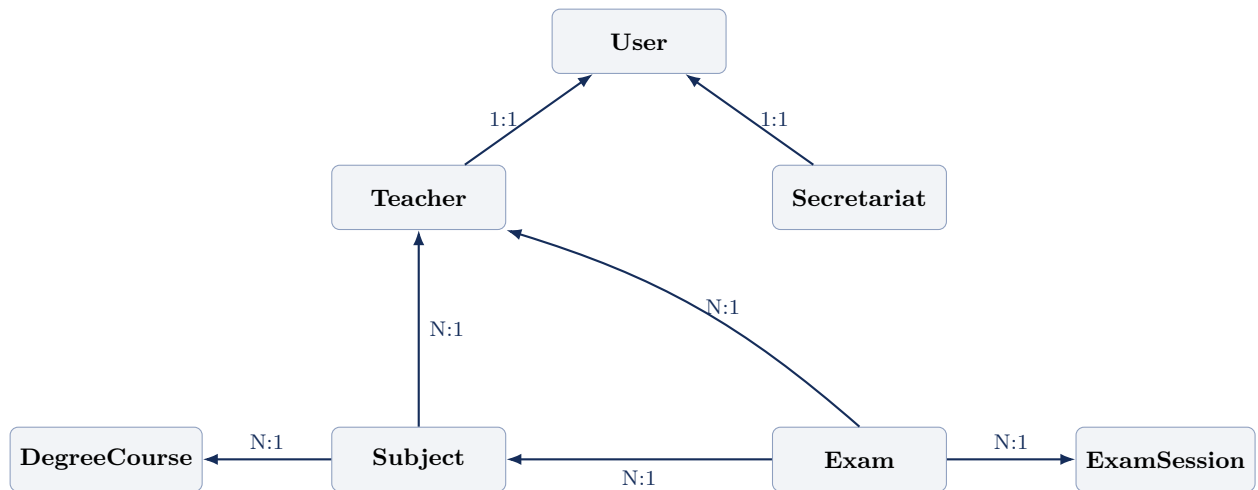


Figura 4.1: Relazioni tra le entità del dominio. Teacher e Secretariat estendono un User (1:1, cascade); Subject appartiene a un DegreeCourse e a un Teacher; Exam collega Subject, ExamSession e Teacher.

- la **chiave primaria** è una colonna `id` auto-generata, ma nel codice è esposta con un nome parlante (`degreeCourseId`, `subjectId`, `examId`...) tramite `@PrimaryGeneratedColumn({ name: 'id' })`;
- le relazioni verso “il padre” sono `@ManyToOne` con `eager: true` (la relazione è sempre caricata insieme all’entità) e `onDelete: 'RESTRICT'` (non si può cancellare il padre se esistono figli);
- i vincoli di unicità “di dominio” sono espressi con `@Unique([...])` o `unique: true` sulla colonna.

4.4.2 User, Teacher, Secretariat

`UserEntity` (tabella `users`) è la base dell’autenticazione: ha `name`, `email` (univoca, è l’identificativo di login), `passwordHash` (hash `bcrypt`, annotato `@Exclude()` per non uscire mai nelle risposte) e `role` (enum `UserRole`, default `DOCENTE`).

`TeacherEntity` e `SecretariatEntity` sono i *profili*: ciascuno ha una relazione `@OneToOne` verso `UserEntity` con `onDelete: 'CASCADE'` (eliminando l’utente cade anche il profilo) ed `eager: true`. La separazione utente/profilo permette di tenere le credenziali in un’unica tabella e i dati specifici del ruolo altrove. Il docente possiede inoltre gli insegnamenti (`@OneToMany` verso `Subject`) e gli appelli (`@OneToMany` verso `Exam`); il segretario non possiede risorse proprie perché amministra quelle altrui.

4.4.3 DegreeCourse e Subject

`DegreeCourseEntity` (degree_courses) ha `name` (univoco), `yearsDuration` (durata in anni, usata per validare l’anno degli insegnamenti) e `department`.

`SubjectEntity` (subjects) ha `name`, `year`, `cfu`, e due `@ManyToOne` obbligatori verso `DegreeCourse` e `Teacher`. Il vincolo `@Unique(['name', 'degreeCourse'])` impedisce due insegnamenti con lo stesso nome nello stesso corso.

```

1 @Entity('subjects')
2 @Unique(['name', 'degreeCourse'])
3 export class SubjectEntity {
4   @PrimaryGeneratedColumn({ name: 'id' }) subjectId: number;
5   @Column() name: string;

```

```
6  @Column() year: number;
7  @Column() cfu: number;
8
9  @ManyToOne(() => DegreeCourseEntity, (dc) => dc.subjects,
10    { nullable: false, eager: true, onDelete: 'RESTRICT' })
11  degreeCourse: DegreeCourseEntity;
12
13  @ManyToOne(() => TeacherEntity, (t) => t.subjects,
14    { nullable: false, eager: true, onDelete: 'RESTRICT' })
15  teacher: TeacherEntity;
16 }
```

Listing 4.4: Vincolo e relazioni di subject.entity.ts (estratto).

4.4.4 ExamSession ed Exam

ExamSessionEntity (exam_sessions) ha name (univoco) e *quattro* date: startDate/endDate (periodo in cui si svolgono gli appelli) e planningStartDate/planningEndDate (finestra entro cui i docenti possono pianificare). Questa distinzione è il cuore della regola “il docente pianifica solo a finestra aperta”.

ExamEntity (exams) ha date, startHour, endHour, type (enum ExamType: orale/scritto), roomType (enum RoomType: aula standard o laboratori specializzati) e tre @ManyToOne verso Subject, ExamSession e Teacher. Il vincolo @Unique(['date', 'subject']) impedisce due appelli della stessa materia nello stesso giorno (le altre regole, come il conflitto “corso+anno+giorno”, sono applicate dal service perché troppo articolate per un solo vincolo SQL).

4.5 Lo script di seed

Il file apps/api/src/seed.ts popola il database con dati di prova coerenti: account (un segretario e due docenti), corsi, insegnamenti e soprattutto tre sessioni *passata*, *corrente* e *futura* con i relativi appelli. Serve perché, attraverso l’API, un esame può essere creato solo dal docente e solo dentro la finestra di pianificazione: dall’interfaccia non si potrebbero quindi inserire appelli di sessioni passate. Lo script apre un *application context* di Nest e scrive direttamente tramite il DataSource di TypeORM (CRUD puro), **bypassando** le regole di dominio dei service.

È **idempotente** (logica *find-or-create* per nome/email): può essere rieseguito senza creare duplicati. Si lancia con `npm run seed` (richiede PostgreSQL avviato e le variabili del `.env`). La password di tutti gli account di prova è Password1!.

Capitolo 5

Sicurezza e autenticazione

L'autenticazione è basata su **token JWT** emessi al login e verificati a ogni richiesta successiva, con **Passport** come libreria di strategie e un sistema di **guard** e **decoratori** per l'autorizzazione per ruolo. La materia è ripartita su tre librerie: `@server/security` (i mattoni riusabili di autorizzazione), `@server/auth` (login, emissione token, strategie) e `@server/users` (gli account e il loro CRUD).

5.1 La libreria `@server/security`

Raccoglie i componenti di autorizzazione condivisi da tutti i moduli.

L'enum `UserRole`. I due ruoli del sistema. Vive qui (e non in `@server/users`) per rompere una dipendenza circolare tra i moduli utenti e sicurezza.

`JwtAuthGuard`. Estende l'`AuthGuard('jwt')` di Passport: protegge una rotta richiedendo un token valido. Estrae il token dall'header `Authorization`, lo verifica e popola `request.user`; senza token valido risponde **401**.

`RolesGuard`. Autorizza per ruolo e va usato *dopo* il `JwtAuthGuard`. Legge, tramite il `Reflector`, i ruoli dichiarati con `@Roles(...)` sulla rotta e consente l'accesso solo se l'utente corrente ne possiede uno; altrimenti **403**.

```
1 canActivate(context: ExecutionContext): boolean {
2   const requiredRoles = this.reflector.getAllAndOverride<UserRole []>(
3     ROLES_KEY, [context.getHandler(), context.getClass()]);
4   if (!requiredRoles || requiredRoles.length === 0) return true; // rotta
      libera
5   const user = context.switchToHttp().getRequest().user;
6   if (!user) throw new ForbiddenException('Utente non presente nella
      richiesta');
7   if (!requiredRoles.includes(user.role))
8     throw new ForbiddenException('Permessi insufficienti');
9   return true;
10 }
```

Listing 5.1: `roles.guard.ts` (estratto).

I decoratori `@Roles(...)` e `@CurrentUser()`. `@Roles` salva i ruoli ammessi nei metadati della rotta (sotto la chiave `ROLES_KEY`) che il `RolesGuard` poi rilegge. `@CurrentUser()` è un *param decorator* che estrae `request.user` e lo passa come argomento al metodo del controller, evitando di leggere a mano la request:

```
1 @Post()
2 @UseGuards(JwtAuthGuard, RolesGuard)
3 @Roles(UserRole.DOCENTE)
4 @ApiBearerAuth()
5 create(@Body() dto: CreateExamDto, @CurrentUser() user: AuthenticatedUser) {
6   return this.service.createOne(dto, user);
7 }
```

Listing 5.2: Uso combinato di guard e decoratori su una rotta.

5.2 La libreria @server/auth

Configura Passport e l'emissione/verifica dei token, ed espone le rotte di login e registrazione.

Il modulo. `ServerAuthModule` importa `ServerUsersModule` (per leggere/creare utenti), `PassportModule` e `JwtModule`; quest'ultimo è configurato con il segreto `SECRET_KEY` e una scadenza dei token di **24 ore**. Registra come provider le due strategie e il service.

Le due strategie Passport.

LocalStrategy

usata al login dal `LocalAuthGuard`. È configurata con `usernameField: 'email'` (l'identificativo è l'email, non lo *username* di default) e delega la verifica delle credenziali al service.

JwtStrategy

usata dal `JwtAuthGuard`. Estrae il token come *Bearer* dall'header, ne verifica la firma con `SECRET_KEY` e rifiuta i token scaduti; dal payload ricostruisce l'oggetto `AuthenticatedUser` che finisce in `request.user`.

Il service. `ServerAuthService` contiene la logica vera:

```
1 async validateUser(email: string, password: string):
2   Promise<AuthenticatedUser> {
3     const user = await this.userService.findByEmail(email);
4     if (!user) throw new UnauthorizedException('Credenziali non valide');
5     const ok = await bcrypt.compare(password, user.passwordHash);
6     if (!ok) throw new UnauthorizedException('Credenziali non valide');
7     const { passwordHash, ...result } = user; // mai esporre l'hash
8     return result;
9   }
10 async login(user: AuthenticatedUser): Promise<AuthResponse> {
11   const payload = { sub: user.id, name: user.name, email: user.email, role:
12     user.role };
13   return { access_token: await this.jwtService.signAsync(payload), user };
```

Listing 5.3: `auth.service.ts` (estratto).

Due aspetti di sicurezza degni di nota:

- **nessuna *user enumeration***: sia per email inesistente sia per password errata viene lanciato lo stesso **401** con lo stesso messaggio “Credenziali non valide”, così un attaccante non può dedurre quali email siano registrate;

- **l'hash della password non esce mai:** viene rimosso con destrutturazione (`const { passwordHash, ...result } = user`) e in più è marcato `@Exclude()` sull'entità.

Il **payload** del JWT è `{ sub, name, email, role }` (la convenzione JWT vuole `sub = subject = id` utente). Le interfacce `JwtPayload`, `AuthenticatedUser` e `AuthResponse` tipizzano rispettivamente il contenuto del token, l'utente a runtime in `request.user` e la risposta di login/registrazione (`{ access_token, user }`).

Le rotte di autenticazione.

POST /auth/login

rotta **pubblica**: il `LocalAuthGuard` valida email+password e popola `req.user`; il controller restituisce token e utente.

POST /auth/register

riservata alla **Segreteria** (`JwtAuthGuard + RolesGuard + @Roles(SEGRETERIA)`). Crea un utente “nudo” (senza profilo) e ne restituisce subito il token. Non è la via normale di onboarding (vedi sotto).

5.3 La libreria @server/users

Gestisce gli account. `ServerUsersService` si occupa dell'hashing (`bcrypt`, 10 *salt rounds*), del controllo di unicità dell'email (**409** se già in uso) e della traduzione di “non trovato” in **404**; delega l'accesso ai dati a `UsersRepository`, un wrapper di puro CRUD sul `Repository<UserEntity>` di `TypeORM`. Il `ServerUsersController` espone il CRUD utenti, quasi tutto riservato alla Segreteria, più la rotta `GET /users/me` che restituisce il payload dell'utente autenticato (`{ id, name, email, role }`).

Regole sulla password. Definite dichiarativamente nel `CreateUserDto` con `class-validator`: minimo 8 caratteri, almeno una lettera maiuscola e almeno un simbolo tra `? ^ ! # @`. I messaggi di errore sono in italiano.

5.4 Il flusso di autenticazione e la policy di onboarding

Mettendo insieme i pezzi, una richiesta autenticata segue questo percorso:

1. il client invia `POST /api/auth/login` con email e password;
2. `LocalAuthGuard`→`LocalStrategy`→`validateUser` verificano le credenziali; in caso positivo `login` firma un JWT e restituisce `{ access_token, user }`;
3. il client conserva il token e lo invia nell'header `Authorization: Bearer <token>` a ogni chiamata successiva;
4. sulle rotte protette il `JwtAuthGuard`→`JwtStrategy` verificano il token e popolano `request.user`; dove serve, il `RolesGuard` controlla anche il ruolo.

Onboarding reale. È importante notare che `POST /auth/register` e `POST /users` creano un utente *senza profilo* e perciò **non** sono la via normale per registrare gli account. La creazione reale avviene via `POST /teachers` e `POST /secretariats` (riservate alla Segreteria): ciascuna crea in modo *atomico* l'utente e il suo profilo, forzando il ruolo lato server e facendo *rollback* dell'utente se la creazione del profilo fallisce. Questo meccanismo è descritto nel capitolo 6.

Capitolo 6

Il pattern a livelli e la logica di dominio

Il cuore applicativo del backend è la libreria `@server/exam-planning`, che contiene tutte le entità di dominio e la loro logica. Ogni entità segue la stessa struttura a livelli; questo capitolo prima descrive il *pattern* in generale, poi entra nelle regole specifiche di ciascuna entità.

6.1 Il pattern a quattro livelli

Ogni feature di dominio è organizzata negli stessi quattro livelli, con responsabilità nette. Questa uniformità rende il codice prevedibile: imparato un modulo, si conoscono tutti.

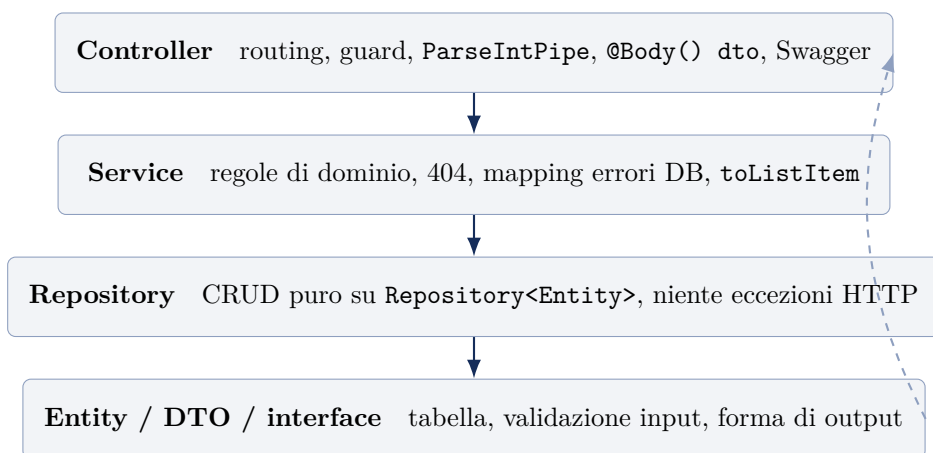


Figura 6.1: I quattro livelli di ogni feature di dominio e il flusso di una richiesta dall'alto verso il basso.

Repository (`xxx.repository.ts`)

classe `@Injectable` che incapsula il `Repository<XxxEntity>` di TypeORM. Solo CRUD puro (`findAll`, `findById`, `createOne`, `updateOne`, `deleteOne`) più eventuali query di supporto. Usa `find/findOne` (non il *QueryBuilder*) così che le relazioni *eager* siano sempre caricate. Non lancia eccezioni HTTP: ritorna `null/boolean` per segnalare assenza o esito.

Service (`xxx.service.ts`)

inietta il repository *custom* (non `@InjectRepository`). Traduce `null` → `NotFoundException`, racchiude le scritture in `try/catch` + `handleDatabaseError`, e possiede le regole di dominio. Espone sempre una forma “piatta” `XxxListItem`, mai l'entità TypeORM: un `toListItem(entity)` privato fa la conversione. Quando una lettura serve internamente

come entità, il service usa un privato `getEntityById` (non il pubblico `findById`, che ora ritorna un `list-item`).

List-item (`interfaces/xxx-list-item.interface.ts`)

l'interfaccia della forma esposta al client (e condivisa col frontend, vedi §6.2). Una per entità, usata sia per la lista sia per il dettaglio.

Controller (`xxx.controller.ts`)

routing + guard (`JwtAuthGuard`, `RolesGuard` + `@Roles`) + `ParseIntPipe` sui parametri `:id`, Swagger minimale (`@ApiTags`, `@ApiBearerAuth`). Usa `@Body()` `dto: XxxDto` (validato dal `ValidationPipe` globale) e *non* annota il tipo di ritorno: eredita il `list-item` dal service.

Perché un `list-item` e non l'entità. Esporre direttamente l'entità TypeORM avrebbe due difetti: trascinerebbe nel JSON le relazioni `eager` (con campi sensibili come `user.passwordHash`) e legherebbe il contratto dell'API alla struttura del database. Il `toListItem` produce invece una forma stabile e "piatta": rinomina `xxxId`→`id`, annida le chiavi esterne come `{ id, name }` e rimuove i campi sensibili. Esempio:

```

1 private toListItem(exam: ExamEntity): ExamListItem {
2   return {
3     id: exam.examId,
4     date: this.toIso(exam.date),           // 'YYYY-MM-DD'
5     startHour: exam.startHour, endHour: exam.endHour,
6     roomType: exam.roomType, type: exam.type,
7     subject: { id: exam.subject.subjectId, name: exam.subject.name },
8     examSession: { id: exam.examSession.examSessionId, name:
9       exam.examSession.name },
10    teacher: { id: exam.teacher.teacherId, name: exam.teacher.user.name },
11  };
12 }

```

Listing 6.1: `toListItem` degli esami (estratto).

6.2 Il barrel `index.ts` e la condivisione col frontend

Il file `libs/server/exam-planning/src/index.ts` è la *superficie pubblica* della libreria: ri-esporta il modulo NestJS *più* tutte le entità, i DTO, gli enum (`ExamType`, `RoomType`) e le interfacce `list-item`. Grazie a questo, il frontend può scrivere

```

1 import type { ExamListItem, CreateExamDto } from '@server/exam-planning';

```

Listing 6.2: Import dei tipi del backend nel frontend.

riusando *gli stessi tipi* del backend. Si usa `import type` per importare solo i *tipi* (cancellati a compile-time) ed evitare di trascinare nel bundle del browser il codice runtime di `class-validator`/Swagger. È un beneficio diretto del monorepo: un unico contratto, tipizzato, condiviso tra i due lati.

6.3 I DTO: validazione e trasformazione dell'input

Il corpo di ogni richiesta di scrittura è descritto da un **DTO** (*Data Transfer Object*): una classe i cui campi portano i decoratori di `class-validator` (es. `@IsString`, `@IsInt`, `@IsEmail`, `@IsOptional`) e gli `@ApiProperty` per Swagger. Il `ValidationPipe` globale li fa rispettare automaticamente, scartando i campi non dichiarati e convertendo i tipi. I messaggi d'errore sono in italiano.

Per evitare duplicazioni, gli `UpdateXxxDto` **non** riscrivono i campi ma derivano dal corrispondente `CreateXxxDto` con i *mapped types* di `@nestjs/swagger`: `PartialType` rende tutti i campi opzionali (così il PATCH può essere parziale), e `OmitType` ne esclude alcuni. Si usano le versioni di `@nestjs/swagger` (non di `@nestjs/mapped-types`) perché preservano anche i metadati Swagger.

```

1 export class CreateDegreeCourseDto {
2   @ApiProperty({ example: 'Ingegneria Informatica', maxLength: 255 })
3   @IsString({ message: 'Il nome del corso di laurea deve essere una stringa di
      testo' })
4   @NotEmpty({ message: 'Il nome del corso di laurea è obbligatorio' })
5   @MaxLength(255) name: string;
6
7   @ApiProperty({ default: 3, required: false })
8   @IsOptional() @IsInt() @Min(1) @Max(6) // opzionale, default 3 sul DB
9   yearsDuration?: number;
10
11   @IsString() @NotEmpty() department: string;
12 }
13
14 // Tutti i campi opzionali
15 export class UpdateDegreeCourseDto extends PartialType(CreateDegreeCourseDto)
16   {}
17
18 // Tutti opzionali E senza 'password' (si cambia per altra via)
19 export class UpdateUserDto extends PartialType(
20   OmitType(CreateUserDto, ['password'] as const)) {}

```

Listing 6.3: DTO Create/Update (estratti).

Un principio di sicurezza importante: i **campi sensibili non compaiono mai** nei DTO rivolti al client. Il role, ad esempio, non è nel `CreateTeacherDto`/`CreateSecretariatDto`: è *forzato lato server* nel service (cfr. la creazione atomica più avanti), così il client non può auto-assegnarsi un ruolo.

6.4 La gestione centralizzata degli errori del database

Le scritture possono fallire per violazioni di vincoli del database. L’helper `handleDatabaseError(error, fallback)` (in `database-error.helper.ts`, tipo di ritorno `never`) traduce i codici d’errore di PostgreSQL nelle eccezioni HTTP appropriate:

Codice PG	Significato	HTTP
23505	unique_violation	409 Conflict
23502	not_null_violation	422 Unprocessable Entity
23503	foreign_key_violation	400 Bad Request
23514	check_violation	422 Unprocessable Entity
23P01	exclusion_violation	409 Conflict
40001 / 40P01	serialization / deadlock	503 Service Unavailable
(altro)	—	500 Internal Server Error

Tabella 6.1: Mappatura codici PostgreSQL → eccezioni HTTP.

Il pattern adottato nei service è: `try { ... } catch (e) { if (e instanceof NotFoundException) throw e; handleDatabaseError(e, '...'); }`. Le eccezioni applicative (`NotFoundException`, `ForbiddenException...`) vengono rilanciate *prima* di delegare all’helper, così non vengono

trasformate per errore in un 500. Avere `never` come tipo di ritorno permette di usarlo come ultima istruzione del `catch` soddisfacendo il controllo di tipo.

Unicità “umana” dei nomi. Un vincolo `UNIQUE` di PostgreSQL confronta i byte esatti: distingue maiuscole e spazi. L’helper `name-normalize.helper.ts` fornisce `normalizeName` (trim + spazi interni collassati, usato in salvataggio) e `nameKey` (chiave case-insensitive per il confronto), così “Programmazione Web” e “programmazione web ” sono trattati come duplicati.

6.5 Le rotte REST del dominio

La tabella riassume gli endpoint principali e il ruolo richiesto. Il prefisso globale `/api` è sottinteso.

Metodo & rotta	Ruolo	Azione
GET <code>/degree-courses</code>	SEGR	elenco corsi
GET <code>/degree-courses/mine</code>	DOC	i corsi del docente
POST/PATCH/DELETE <code>/degree-courses</code>	SEGR	CRUD corsi
GET <code>/subjects</code>	SEGR/DOC	elenco insegnamenti (con filtri)
GET <code>/subjects/mine</code>	DOC	gli insegnamenti del docente
POST/PATCH/DELETE <code>/subjects</code>	SEGR	CRUD insegnamenti
GET <code>/exam-sessions</code>	SEGR/DOC	elenco sessioni (lettura per entrambi)
POST/PATCH/DELETE <code>/exam-sessions</code>	SEGR	CRUD sessioni
GET <code>/exams</code>	SEGR	tutti gli appelli
GET <code>/exams/mine</code>	DOC	gli appelli del docente
GET <code>/exams/by-course-year</code>	DOC	appelli per corso+anno (calendario)
POST <code>/exams</code>	DOC	crea un appello
PATCH/DELETE <code>/exams/:id</code>	SEGR/DOC	modifica/elimina (regole di proprietà)
GET <code>/teachers</code> , POST <code>/teachers</code>	SEGR	elenco / crea docente
GET <code>/teachers/me</code>	DOC	il proprio profilo
PATCH/DELETE <code>/teachers/:id</code>	SEGR/DOC	modifica/elimina (self per il docente)
GET/POST <code>/secretariats</code>	SEGR	elenco / crea segretario
PATCH/DELETE <code>/secretariats/:id</code>	SEGR	solo il proprio profilo (self-only)

Tabella 6.2: Endpoint REST del dominio e ruolo richiesto.

6.6 Logica per entità

6.6.1 Corsi di laurea

Gestiti dalla Segreteria. Il service valida l’unicità “umana” del nome (via `nameKey`) oltre al vincolo del DB, e **impedisce l’eliminazione** di un corso che abbia insegnamenti collegati: poiché la FK è `RESTRICT`, controlla *proattivamente* il conteggio dei figli e risponde con un **409** dal messaggio chiaro, invece del 400 generico da violazione di foreign key.

6.6.2 Insegnamenti

Il service coordina più repository. In creazione/aggiornamento: risolve le FK (corso, docente) con 404 se assenti; valida che l’`year` sia compreso tra 1 e `yearsDuration` del corso; verifica l’unicità del nome *all’interno* dello stesso corso. L’aggiornamento rivaluta queste regole sui valori *risultanti* (mix tra DTO e valori esistenti). Anche qui l’eliminazione è bloccata (409) se

esistono appelli collegati. La lettura supporta filtri per corso e/o docente, e una rotta /mine per il docente.

6.6.3 Sessioni d'esame

Gestite dalla Segreteria. La validazione delle date impone: `startDate < endDate`, `planningStartDate < planningEndDate` e `planningEndDate ≤ startDate` (la pianificazione si chiude prima che la sessione inizi). Inoltre i *periodi* [`startDate`, `endDate`] di due sessioni **non possono sovrapporsi** (le finestre di pianificazione invece sì). Le date ISO sono confrontate direttamente come stringhe. L'eliminazione è bloccata (409) se la sessione ha appelli.

6.6.4 Appelli (la logica più ricca)

Il service degli esami coordina quattro repository (esami, materie, sessioni, docenti) e concentra le regole più articolate.

Creazione (solo Docente). Risolve materia, sessione e il profilo docente dell'utente corrente; verifica che la materia sia *del* docente (altrimenti 403); applica `validateExam` e salva, forzando il docente all'utente autenticato.

La validazione `validateExam`. È applicata sia in creazione sia in modifica, e lancia alla prima regola violata:

1. `startHour < endHour` (altrimenti 400);
2. la data dev'essere compresa tra `startDate` e `endDate` della sessione (400);
3. se l'utente è un Docente, "oggi" deve cadere nella finestra di pianificazione [`planningStartDate`, `planningEndDate`] (altrimenti 403);
4. la data non può essere sabato o domenica (400). Si usa `getUTCDay` sulla data ISO per non sfasare il giorno della settimana col fuso del server;
5. nessun **conflitto corso+anno+giorno**: non può esistere un altro appello nello stesso giorno per lo stesso anno dello stesso corso di laurea (409). In modifica, l'esame corrente è escluso dal confronto;
6. la stessa materia non può avere due appelli nella stessa sessione (409).

Modifica ed eliminazione. La Segreteria può modificare/eliminare qualsiasi appello; il Docente solo i propri (403 altrimenti) e **non** se l'appello è già passato (un esame è "passato" se la sua data è precedente a oggi). La modifica rivaluta `validateExam` sui valori risultanti.

La rotta `by-course-year`. Restituisce tutti gli appelli di un dato corso+anno, di *qualsiasi* docente. Serve al calendario del docente sia per colorare di rosso i giorni "occupati" dalla regola di conflitto, sia per mostrare *quali* esami occupano un giorno in cui non può inserirne uno proprio (anche se non ne è il titolare).

6.6.5 Docenti e segretari (creazione atomica e *self-only*)

La creazione di un docente o segretario coinvolge due tabelle (utente e profilo) e dev'essere atomica:

```
1 async createOne(dto: CreateTeacherDto): Promise<TeacherListItem> {  
2   dto.name = normalizeName(dto.name);  
3   const user = await this.userService.create({ ...dto, role: UserRole.DOCENTE  
4     });  
5   try {  
6     return this.toListItem(await this.repository.createOne(user));  
7   } catch (error) {  
8     await this.userService.removeUser(user.id).catch(() => undefined); //  
9     rollback  
10    handleDatabaseError(error, 'Errore durante la creazione del docente');  
11  }  
12 }
```

Listing 6.4: Creazione atomica di un docente con rollback.

Il ruolo è **forzato lato server** (role: UserRole.DOCENTE / SEGRETERIA), non arriva mai dal client. Se la creazione del profilo fallisce, l'utente appena creato viene rimosso, evitando account “orfani”.

Sulle modifiche e cancellazioni valgono i controlli di proprietà:

- un **Docente** può vedere/modificare/eliminare solo il proprio profilo e non può cambiarsi l'email; non è eliminabile se ha insegnamenti o appelli collegati (409). La Segreteria può gestire qualsiasi docente;
- un **Segretario** è *self-only* su tutta la linea: può vedere/modificare/eliminare solo il proprio profilo, e nemmeno cambiarsi l'email. La Segreteria può crearne e elencarli.

Capitolo 7

Il frontend React

Il frontend (apps/ui) è una **SPA** React costruita con Vite, con routing client tramite React Router e stile Tailwind CSS. Segue la mappatura WebML→React del corso: le *pagine* diventano componenti `*.page.tsx`, le *operazioni* verso il backend stanno in `*.api.ts`, le *aree landmark* sono le voci della navbar.

7.1 Organizzazione del codice

Il codice è organizzato **per feature**: ogni entità di dominio ha una cartella sotto `src/features/` con le sue pagine e il suo client API. La struttura tipica di una feature è:

- `xxx.api.ts` — funzioni che chiamano il backend (fetch + gestione errori);
- `xxx.page.tsx` — la pagina-lista;
- `create-xxx.page.tsx`, `edit-xxx.page.tsx` — i form di creazione e modifica.

Accanto alle feature di dominio ci sono cartelle trasversali: `auth/` (login, logout, rotta protetta, client di autenticazione), `shared/` (infrastruttura HTTP comune), `layouts/` (il guscio con la navbar), `calendar/` (il componente calendario) e `dashboard/` (la home).

7.2 Avvio e routing

Il file `main.tsx` monta l'app dentro un `<BrowserRouter>`. Il componente `App` (`app/app.tsx`) dichiara tutte le rotte:

```
1 <Routes>
2   <Route path="/login" element={<LoginPage />} />    {/* pubblica */}
3   <Route path="/logout" element={<LogoutPage />} />
4
5   {/* Gruppo protetto: ProtectedRoute (richiede token) + AppLayout (navbar) */}
6   <Route element={<ProtectedRoute><AppLayout /></ProtectedRoute>}>
7     <Route index element={<DashboardPage />} />      {/* home '/' */}
8     <Route path="/exams" element={<ExamsPage />} />
9     <Route path="/exams/new" element={<CreateExamPage />} />
10    <Route path="/exams/:id/edit" element={<EditExamPage />} />
11    {/* ...le altre entità... */}
12    <Route path="/me" element={<ProfilePage />} />
13  </Route>
14
15  <Route path="*" element={<Navigate to="/" replace />} /> {/* fallback */}
16 </Routes>
```

Listing 7.1: Struttura delle rotte (estratto di `app.tsx`).

Si nota la struttura a **due livelli**: le rotte pubbliche (`/login`, `/logout`) e un **gruppo protetto** annidato sotto `ProtectedRoute + AppLayout`. Tutte le pagine autenticate sono figlie di questo gruppo e vengono iniettate nell'`<Outlet/>` del layout; qualsiasi rotta sconosciuta rimanda alla home.

La rotta protetta. `ProtectedRoute` è semplicissima: se in `localStorage` non c'è il token, reindirizza a `/login`; altrimenti mostra i figli.

```
1 export function ProtectedRoute({ children }: { children: ReactNode }) {
2   const token = localStorage.getItem('access_token');
3   if (!token) return <Navigate to="/login" replace />;
4   return children;
5 }
```

Listing 7.2: `protected-route.tsx`.

7.3 L'infrastruttura HTTP condivisa

Tutte le chiamate passano da `features/shared/utils.api.ts`, che centralizza tre cose: l'URL base del backend, gli header autenticati e la gestione degli errori. Così ogni `*.api.ts` le riusa senza duplicarle.

```
1 export const API_URL = 'http://localhost:3333/api';
2
3 export function getAuthHeaders(): HeadersInit {
4   const token = localStorage.getItem('access_token');
5   return { 'Content-Type': 'application/json',
6     ...(token ? { Authorization: `Bearer ${token}` } : {}) };
7 }
8
9 export async function handleApiError(response: Response): Promise<never> {
10   let message = `Errore ${response.status}`;
11   try {
12     const body = await response.json();
13     if (Array.isArray(body?.message)) message = body.message.join('\n');
14     else if (typeof body?.message === 'string') message = body.message;
15   } catch { /* corpo non-JSON: si tiene il default */ }
16   throw new Error(message);
17 }
```

Listing 7.3: `shared/utils.api.ts` (estratto).

`handleApiError` merita attenzione: il backend NestJS restituisce un corpo `{ message: string | string[] }`. Quando `message` è un *array* (tipico degli errori di `class-validator`, un messaggio per regola), li unisce su più righe; quando è una stringa, la usa così com'è. In entrambi i casi il messaggio mostrato all'utente è quello *italiano* prodotto dal backend.

7.4 L'autenticazione lato client

`features/auth/auth.api.ts` implementa il login e le utilità sull'utente corrente. Al login salva token e utente in `localStorage`; espone `getCurrentUser()` (lettura sincrona, senza rete) usato dalle pagine per conoscere il ruolo, e `logout()` che ripulisce lo storage.

```
1 export async function login(email: string, password: string):
2   Promise<CurrentUser> {
3   const response = await fetch(`${API_URL}/auth/login`, {
4     method: 'POST', headers: { 'Content-Type': 'application/json' },
```

```

4     body: JSON.stringify({ email, password }),
5   });
6   if (!response.ok) await handleApiError(response);
7   const data = await response.json();
8   localStorage.setItem('access_token', data.access_token);
9   localStorage.setItem('current_user', JSON.stringify(data.user));
10  return data.user;
11 }

```

Listing 7.4: login (estratto di auth.api.ts).

La LoginPage è un *form controllato* (i valori dei campi vivono nello *state* React): al submit chiama login e, in caso di successo, naviga alla home; in caso di errore mostra il messaggio del backend.

7.5 Il layout e la navbar

AppLayout è il guscio delle pagine autenticate: una navbar fissa in alto e l'<Outlet/> per la pagina corrente. Le voci di menu sono dichiarate in un array; alcune sono marcate `segreteriaOnly` e compaiono solo per la Segreteria, applicando lato UI la stessa divisione di permessi del backend:

```

1  const navItems = [
2    { to: '/exams', label: 'Esami' },
3    { to: '/exam-sessions', label: 'Sessioni' },
4    { to: '/subjects', label: 'Insegnamenti' },
5    { to: '/degree-courses', label: 'Corsi di laurea' },
6    { to: '/teachers', label: 'Docenti', segreteriaOnly: true },
7    { to: '/secretariats', label: 'Segretari', segreteriaOnly: true },
8  ];
9  const isSegreteria = user?.role === 'SEGRETARIA';
10 const visibleNavItems = navItems.filter(i => !i.segreteriaOnly ||
    isSegreteria);

```

Listing 7.5: Voci di navbar e filtro per ruolo (estratto).

La navbar mostra anche nome e ruolo dell'utente (link all'area personale /me) e un pulsante di logout con popup di conferma.

7.6 Il pattern api/page in dettaglio

Ogni client di feature è costruito sopra `utils.api` e importa i *tipi* dal backend. Esempio (esami):

```

1  import { API_URL, getAuthHeaders, handleApiError } from '../shared/utils.api';
2  import type { ExamListItem, CreateExamDto, UpdateExamDto } from
    '@server/exam-planning';
3
4  const RESOURCE = `${API_URL}/exams`;
5
6  export async function fetchMyExams(): Promise<ExamListItem[]> {           //
    DOCENTE
7    const r = await fetch(`${RESOURCE}/mine`, { headers: getAuthHeaders() });
8    if (!r.ok) await handleApiError(r);
9    return r.json();
10 }
11 export async function createExam(dto: CreateExamDto): Promise<ExamListItem> {
12   const r = await fetch(RESOURCE, { method: 'POST',
13     headers: getAuthHeaders(), body: JSON.stringify(dto) });
14   if (!r.ok) await handleApiError(r);
15   return r.json();

```

16 }

Listing 7.6: exams.api.ts (estratto).

La pagina-lista carica i dati al montaggio con `useEffect` (il “link automatico” del WebML), tiene i risultati nello *state*, e **adatta il proprio comportamento al ruolo**. Nella `ExamsPage`, ad esempio, la Segreteria vede tutti gli esami (`fetchExams`) mentre il Docente vede solo i propri (`fetchMyExams`); solo il Docente vede il pulsante “Nuovo esame”; gli esami passati del Docente non mostrano i pulsanti di modifica/elimina, rispecchiando il vincolo del backend.

```

1 useEffect(() => {
2   const loadExams = isSegreteria ? fetchExams : fetchMyExams;
3   Promise.all([loadExams(), fetchExamSessions()])
4     .then(([exams, allSessions]) => { setItems(exams);
        setSessions(allSessions); })
5     .catch((err) => setError(err.message))
6     .finally(() => setLoading(false));
7 }, [isSegreteria]);

```

Listing 7.7: Caricamento condizionato al ruolo (estratto di exams.page.tsx).

7.7 I form di creazione e modifica

Ogni feature ha due pagine-form, `create-xxx.page.tsx` e `edit-xxx.page.tsx`, che condividono lo stesso schema. I campi sono **input controllati** (il valore vive nello *state* React) e le chiavi esterne si scelgono con un `<select>` popolato da una chiamata API.

Le opzioni dei select

vengono caricate al montaggio con `useEffect` e, dove serve, in modo *adattivo al ruolo*. Nel form di modifica di un esame, ad esempio, le materie selezionabili sono tutte (`fetchSubjects`) per la Segreteria e solo le proprie (`fetchMySubjects`) per il Docente.

La creazione

può *precompilare* i campi da parametri di query (`useSearchParams`): è così che il clic su un giorno verde del calendario apre `/exams/new?subjectId=&date=&examSessionId=` con materia, data e sessione già impostate.

La modifica

legge l’`id` dalla rotta (`useParams`) e in un `useEffect` chiama `fetchXxxById` per **precompilare** il form con i valori correnti.

Il submit

previene il reload nativo, converte le stringhe dei `<select>` in numeri (`Number(...)`), invia il DTO via API e in caso di successo naviga alla lista; gli errori del backend (in italiano) sono mostrati sotto il form. Le **regole di dominio restano del backend**: il form non le duplica, si limita a mostrare l’eventuale messaggio.

```

1 const { id } = useParams<{ id: string }>();
2 const examId = Number(id);
3
4 useEffect(() => {                                     // precompila leggendo l'esame
5   fetchExamById(examId).then((exam) => {
6     setSubjectId(String(exam.subject.id));
7     setExamSessionId(String(exam.examSession.id));

```



```

8     setDate(exam.date); setStartHour(exam.startHour); setEndHour(exam.endHour);
9     setType(exam.type); setRoomType(exam.roomType);
10  }).catch((err) => setError(err.message)).finally(() => setLoading(false));
11 }, [examId]);
12
13 async function handleSubmit(e: React.FormEvent) {
14     e.preventDefault();
15     try {
16         await updateExam(examId, { subjectId: Number(subjectId),
17                               examSessionId: Number(examSessionId), date, startHour, endHour, type,
18                               roomType });
19         navigate('/exams'); // torna alla lista
20     } catch (err) { setError(err.message); } // mostra l'errore del backend
21 }

```

Listing 7.8: Precompilazione e submit del form di modifica (estratto di edit-exam.page.tsx).

7.8 La dashboard e il calendario

La home (/) è una **dashboard** che cambia in base al ruolo (DashboardPage sceglie tra SegreteriaDashboard e DocenteDashboard). Entrambe mostrano alcune statistiche di sintesi e un **calendario mensile** condiviso.

Il calendario è un componente “stupido”. PlanningCalendar (su react-big-calendar, localizzato in italiano con date-fns) non contiene logica di dominio: riceve gli *eventi* da mostrare e due funzioni dayColor(iso) e dayRing(iso) che, per ogni giorno, restituiscono colore di sfondo ed eventuale bordo. La logica “che colore ha un giorno” vive nella dashboard, che la calcola dallo stato del dominio.

Vista Docente. Scegliendo una propria materia da un menu a tendina, il calendario “evolve”: i giorni in cui può inserire un appello diventano **verdi** (clic → form di creazione precompilato con data, materia e sessione), quelli bloccati **rossi**. Un giorno è bloccato se è weekend, se la pianificazione non è aperta, se c’è già un appello per quel corso+anno (regola di conflitto) o se la materia è già pianificata in quella sessione. Cliccando un giorno rosso, un riquadro mostra *quali* esami lo occupano (anche di altri docenti, recuperati con fetchCourseYearExams). I colori *rispecchiano* le regole, ma la decisione autoritativa resta del backend: alla creazione effettiva è il service a validare.

Vista Segreteria. Mostra le finestre di pianificazione (viola) e i periodi di sessione (azzurro) colorati, con gli esami come eventi; cliccando un giorno si vedono gli appelli di quella data. Un bordo viola sovrapposto segnala i giorni che sono insieme periodo di sessione e finestra di pianificazione.

```

1  const blocked =
2    isWeekend(iso) ||
3    today < session.planningStartDate || // pianificazione non ancora aperta
4    today > session.planningEndDate || // pianificazione chiusa
5    occupiedDates.includes(iso) || // conflitto corso+anno+giorno
6    myExams.some(e => e.subject.id === selectedSubject.id
7                      && e.examSession.id === session.id); // materia già
                        pianificata
8  return blocked ? COLORS.blocked : COLORS.available;

```

Listing 7.9: Calcolo del colore di un giorno, vista Docente (estratto).

7.9 Note sullo stato e sulla cache

Lo stato dell'autenticazione è volutamente semplice: token e utente in `localStorage`, letti in modo sincrono. Non c'è uno store globale (Redux o simili): ogni pagina carica i propri dati con `useEffect` e li tiene nel proprio *state* locale. Per evitare che il browser mostri dati obsoleti, il backend invia `Cache-Control: no-store` su ogni risposta (cfr. capitolo 4); alcune azioni applicano inoltre un aggiornamento *ottimistico* dell'elenco (ad es. rimuovendo subito la riga eliminata senza rifare la fetch).

Capitolo 8

Test e qualità del codice

Il progetto adotta tre livelli di test, eseguiti tramite Nx: **unit test** con Jest sui singoli service/controller, **test d'integrazione** dell'API (api-e2e) e **test end-to-end** della UI con Playwright (ui-e2e). I file di test sono *co-locali* accanto al codice che verificano, con estensione `.spec.ts`.

8.1 Unit test dei service

Sono i test più significativi: verificano le *regole di dominio* in isolamento. Si usa il modulo di testing di NestJS (`Test.createTestingModule`) per costruire un'istanza del service con i suoi repository **sostituiti da mock** (`useValue`), così il test non tocca il database e può simulare qualsiasi situazione.

```
1 beforeEach(async () => {
2   degreeCoursesRepository = { deleteOne: jest.fn() };
3   subjectsRepository = { countByDegreeCourse: jest.fn() };
4   const module = await Test.createTestingModule({
5     providers: [
6       DegreeCoursesService,
7       { provide: DegreeCoursesRepository, useValue: degreeCoursesRepository },
8       { provide: TeachersRepository, useValue: {} },
9       { provide: SubjectsRepository, useValue: subjectsRepository },
10    ],
11  }).compile();
12  service = module.get(DegreeCoursesService);
13 });
14
15 it('rifiuta con 409 se il corso ha insegnamenti collegati, senza eliminare',
16   async () => {
17     subjectsRepository.countByDegreeCourse.mockResolvedValue(2);
18     await expect(service.deleteOne(1)).rejects.toBeInstanceOf(ConflictException);
19     expect(degreeCoursesRepository.deleteOne).not.toHaveBeenCalled();
20   });
```

Listing 8.1: Unit test di una regola di dominio (estratto di `degree-courses.service.spec.ts`).

L'esempio cattura l'essenza della verifica: dato un corso con due insegnamenti collegati (mock di `countByDegreeCourse`), il service deve lanciare `ConflictException` (409) e *non* chiamare `deleteOne`. È il controllo proattivo descritto nel capitolo 6, qui dimostrato automaticamente.

8.2 Smoke test dei controller

Per i controller ci sono *smoke test* che ne verificano la corretta istanziazione con il service mockato (`expect(controller).toBeDefined()`). Confermano che il cablaggio della dependency

injection e dei decorator di routing è valido, senza riesercitare la logica (già coperta dai test del service).

8.3 Test d'integrazione ed end-to-end

apps/api-e2e

test d'integrazione dell'API basati su Jest, che esercitano l'applicazione attraverso vere richieste HTTP.

apps/ui-e2e

test end-to-end della UI con **Playwright**, che guidano un browser reale simulando l'interazione dell'utente.

8.4 Esecuzione

Comando	Effetto
<code>npx nx run-many -t test</code>	tutti gli unit test del workspace
<code>npx nx test @server/exam-planning</code>	i test di una singola libreria
<code>npx nx test @server/exam-planning - exams.service</code>	un singolo file di test
<code>npx nx test @server/exam-planning - -t "should create an exam"</code>	un singolo caso
<code>npx nx e2e api-e2e</code>	test d'integrazione dell'API
<code>npx nx e2e ui-e2e</code>	test end-to-end della UI

Tabella 8.1: Comandi per l'esecuzione dei test.

Oltre ai test automatici, il progetto è stato verificato manualmente nel browser per ciascun ruolo (esiste una `TEST-CHECKLIST.md` che elenca gli scenari per Segreteria e Docente), e con *smoke test* dell'API contro un'istanza reale di PostgreSQL.

Capitolo 9

Interazione end-to-end, WebML e procedure

Questo capitolo conclusivo ricostruisce come tutte le componenti collaborano in una richiesta reale, riepiloga la mappatura WebML→React e raccoglie i comandi per costruire, avviare e testare il progetto.

9.1 Una richiesta dal clic alla risposta

Seguiamo il caso più ricco: un **Docente crea un appello** cliccando un giorno verde nel calendario. La figura 9.1 mostra la sequenza; sotto, i passaggi commentati.

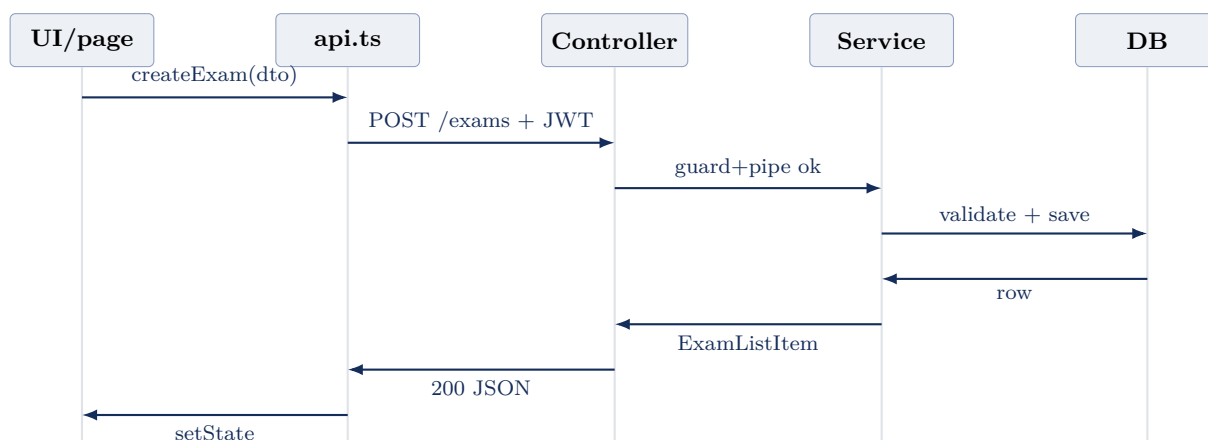


Figura 9.1: Sequenza di una creazione di appello: il frontend chiama l’API, il controller applica guard e pipe, il service valida le regole e salva, la risposta “piatta” risale fino allo stato della UI.

1. **UI** — il clic su un giorno verde naviga al form `/exams/new` con i parametri precompilati (materia, data, sessione); al submit la pagina chiama `createExam(dto)` di `exams.api.ts`.
2. **api.ts** — esegue `fetch` su `POST /api/exams` con header `Authorization: Bearer <token>` e il DTO come corpo JSON.
3. **Controller** — la richiesta attraversa `JwtAuthGuard` (token valido?), `RolesGuard + @Roles(DOCENTE)` (ruolo giusto?) e il `ValidationPipe` globale (il DTO rispetta i decoratori `class-validator`?). Se uno fallisce, la risposta è 401/403/400 e il service non viene mai raggiunto.

4. **Service** — risolve materia/sessione/docente, verifica la proprietà della materia, applica `validateExam` (finestra di pianificazione, niente weekend, conflitto corso+anno+giorno, ecc.) e salva tramite il repository, forzando il docente all'utente autenticato.
5. **Repository/DB** — inserisce la riga; un'eventuale violazione di vincolo risale come `QueryFailedError` e viene tradotta da `handleDatabaseError` nel giusto codice HTTP.
6. **Risposta** — il service converte l'entità in `ExamListItem` (forma piatta, senza dati sensibili); il controller la restituisce come JSON.
7. **UI** — `api.ts` risolve la `Promise`, la pagina aggiorna lo `state` e la lista riflette il nuovo appello.

In sintesi, le responsabilità sono nette: *autenticazione/autorizzazione e forma dell'input* ai guard/pipe del controller; *regole di dominio* al service; *accesso ai dati* al repository; *presentazione e interazione* alla UI. Il contratto tra i due lati è garantito dai tipi condivisi (`ExamListItem`, `CreateExamDto`) della libreria di dominio.

9.2 La mappatura WebML → React

Il frontend non è stato improvvisato: deriva direttamente dal progetto dell'ipertesto WebML. La corrispondenza operativa completa tra primitive WebML e costrutti React (site view, aree landmark, pagine, unità di contenuto, link contestuali, primitive CUD e di login) è riportata nella tabella 3.3 del capitolo 3; qui basti ricordare i capisaldi: *site view* → rendering per ruolo; *pagina landmark* → voce di navbar; *pagina* → componente `*.page.tsx`; *operazioni CUD* → funzioni in `*.api.ts`; *link automatico* → `useEffect` al montaggio.

9.3 Comandi per build, avvio e test

Il package manager è **npm**; i task passano per Nx. I comandi principali:

Comando	Effetto
<code>npm run start:api</code>	avvia l'API (nx <code>serve api</code> , porta da <code>.env</code>)
<code>npx nx serve ui</code>	avvia il frontend (porta 4200)
<code>npm run dev</code>	avvia API e UI insieme
<code>npm run seed</code>	popola il database con dati di prova
<code>npx nx build api ui</code>	build di produzione
<code>npx nx run-many -t test</code>	esegue tutti i test
<code>npx nx test @server/exam-planning</code>	test di una singola libreria
<code>npx nx run-many -t lint</code>	lint di tutti i progetti
<code>npx nx e2e api-e2e ui-e2e</code>	test end-to-end (Jest / Playwright)
<code>npx nx sync</code>	riallinea i <i>project references</i> dei tsconfig

Tabella 9.1: Comandi principali del workspace.

Prima dell'avvio va copiato e compilato il file `.env` con le variabili del capitolo 4. Con l'API attiva, la documentazione Swagger interattiva è su `http://localhost:<PORT>/api/docs` (es. porta 3333). Una nota operativa: il `watch` di `nx serve api` non ricarica le modifiche fatte dentro `libs/`; per provarle occorre riavviare l'API.

9.4 Conclusioni

Il progetto realizza un sistema di pianificazione degli appelli completo, costruito con uno stack moderno e coerente con le metodologie del corso. Gli elementi qualificanti sono:

- un'**architettura a monorepo** che condivide tipi e contratti tra backend e frontend, eliminando le duplicazioni;
- un **pattern a livelli** uniforme (Repository / Service / Controller / DTO) che separa nettamente accesso ai dati, regole di dominio e trasporto HTTP;
- una **sicurezza** basata su JWT e su un sistema di guard/decoratori riusabili, con autorizzazione per ruolo e controlli di proprietà fini;
- regole di dominio **autoritative lato server** (finestre di pianificazione, conflitti, vincoli temporali) che il frontend rispecchia ma non sostituisce;
- un **frontend** progettato con il metodo WebML, organizzato per feature e adattivo al ruolo dell'utente.

La stessa struttura, separando il dominio dall'infrastruttura, è facilmente estendibile: aggiungere una nuova entità significa replicare il pattern a quattro livelli e registrarla nel modulo, mentre l'autenticazione, la gestione errori e l'infrastruttura del frontend restano invariate.

Appendice A

Considerazioni di produzione

Il progetto è realizzato e configurato per lo **sviluppo** e per scopi didattici. Questa appendice raccoglie, in modo onesto, gli aspetti che andrebbero rivisti prima di un eventuale rilascio in **produzione**: per ciascuno si indica lo *stato attuale* nel codice e la *modifica* consigliata. Non sono difetti del progetto rispetto ai suoi obiettivi, ma il naturale divario tra un ambiente di sviluppo e uno di esercizio.

A.1 Schema del database: da `synchronize` alle migrazioni

Stato attuale. `DatabaseModule` usa `synchronize: true`: all'avvio TypeORM allinea automaticamente lo schema alle entità.

In produzione. `synchronize` va **disattivato**: può alterare o eliminare colonne/tabelle in modo distruttivo. Lo si sostituisce con le **migrazioni** TypeORM — script versionati che descrivono le variazioni di schema in modo controllato e reversibile, eseguiti in fase di deploy. Così le modifiche al modello dati diventano tracciabili e sicure.

A.2 Gestione dei segreti

Stato attuale. I parametri sensibili (`SECRET_KEY`, credenziali del database) sono letti dal file `.env`; `requireEnv` fa già fallire l'avvio se mancano.

In produzione. Il `.env` non va versionato e i segreti andrebbero forniti da un *secret manager* (variabili d'ambiente del runtime, vault, ecc.). La `SECRET_KEY` dev'essere lunga e casuale, diversa per ogni ambiente, e ruotabile. Anche l'utente del database dovrebbe avere i *minimi privilegi* necessari.

A.3 Autenticazione: scadenza e conservazione del token

Stato attuale. Il JWT dura 24 ore e, lato client, è conservato in `localStorage`; non esiste un meccanismo di *refresh*.

In produzione. Si possono introdurre token di accesso a vita breve più un *refresh token*, e valutare la conservazione in un cookie `httpOnly/Secure` invece che in `localStorage`, che è esposto agli attacchi XSS. Andrebbe inoltre aggiunta una protezione contro il *brute-force* sul login (*rate limiting*, ad es. con `@nestjs/throttler`).

A.4 Trasporto, intestazioni e CORS

Stato attuale. Il CORS è già ristretto a un'origine configurabile (`CORS_ORIGIN`); ogni risposta porta `Cache-Control: no-store`.

In produzione. Tutto il traffico va servito su **HTTPS**. Conviene aggiungere intestazioni di sicurezza (ad es. con **helmet**) e restringere il **no-store** alle sole rotte che lo richiedono, abilitando una cache appropriata per le risorse statiche del frontend.

A.5 Esposizione della documentazione API

Stato attuale. Swagger è sempre attivo su `/api/docs`.

In produzione. L'interfaccia Swagger andrebbe **disabilitata** oppure protetta da autenticazione, per non esporre pubblicamente la mappa completa degli endpoint.

A.6 Build, esercizio e osservabilità

Stato attuale. In sviluppo si usano i *dev server* (Vite per la UI, webpack in *watch* per l'API).

In produzione. Si producono i *bundle* ottimizzati (`nx build api` e `nx build ui`); il frontend statico viene servito da un *web server*/CDN dietro un *reverse proxy*, mentre l'API gira come processo gestito. Andrebbero aggiunti *logging* strutturato, *health check* e un minimo di monitoraggio. La gestione degli errori già evita di esporre dettagli interni: `handleDatabaseError` traduce i codici del database in messaggi controllati, senza rivelare la struttura sottostante.

A.7 Riepilogo

Aspetto	Sviluppo (attuale)	Produzione (consigliato)
Schema DB	<code>synchronize: true</code>	migrazioni versionate
Segreti	file <code>.env</code>	secret manager, segreti forti
Token	JWT 24h in <code>localStorage</code>	access+refresh, cookie <code>httpOnly</code>
Login	nessun limite	<i>rate limiting</i> anti brute-force
Trasporto	HTTP locale	HTTPS + helmet
Swagger	sempre attivo	disattivato o protetto
Esercizio	dev server	build ottimizzata + reverse proxy + log

Tabella A.1: Sintesi delle differenze tra configurazione di sviluppo e di produzione.